PROJECT PRESENTATION

SafeScroll

Industry Guide Presentation Script

On-device, privacy-first content moderation for the web

Full presentation: 22-25 minutes

Includes demonstration cues, evaluation, limitations, future scope, and viva answers

Prepared 9 October 2026

Contents

Use the full script as written, or follow the 12-15 minute route noted at the beginning.

Opening 4

1. Problem statement 4

2. Project objectives 4

3. High-level architecture 5

4. How SafeScroll detects content on a live page 5

5. The moderation pipeline 6

- Layer 1: deterministic lexicon . . . 6
- Layer 2: text-toxicity model . . . 6
- Layer 3: unsafe-image model . . . 6
- Layer 4: violence, gore, and weapon model . . . 7
- Layer 5: OCR for meme text . . . 7
- Final fusion . . . 7

6. Fast two-stage user experience 7

7. User interface and controls 8

8. Privacy, security, and local storage 8

9. Important engineering challenges and solutions 8

10. Evaluation and verification 9

11. Limitations 9

12. Future scope 10

13. Demonstration script 10

14. Conclusion 11

Likely Questions and Suggested Answers 12

Why did you choose a browser extension? 12

Why run the models locally instead of on a server? 12

Why use an offscreen document? 12

Why use both a lexicon and a machine-learning model? 12

Why is the fusion rule an OR condition? 12

How do you control false positives? 12

What happens when a model fails to load? 12

How does SafeScroll avoid slowing down scrolling? 12

What is quantization? 13

Does the extension work completely offline? 13

Is the Child Lock secure? 13

Why did the Hinglish experiments fail? 13

What is the main innovation in this project? 13

How would you deploy this as a real product? 13

60-second backup summary 13

Full duration: approximately 22-25 minutes, including the demonstration but excluding questions **12-15 minute route:** present Sections 1-8, summarize Sections 9-12 in two minutes, then use the demonstration and conclusion **How to use:** Replace the bracketed details before presenting. The script uses "I"; change it to "we" if this is a team project.

Opening

Good morning/afternoon, sir/ma'am.

My name is **[your name]**, and today I am presenting my project, **SafeScroll**.

SafeScroll is a privacy-first Chrome extension that detects harmful content while a person browses the web. It can identify toxic or abusive text, potentially unsafe images, violence, gore, weapons, and toxic text embedded inside meme images. Instead of removing the whole page or blocking the website, it selectively blurs the affected text or image and lets the user decide what to do next.

The key point is that the machine-learning inference runs on the user's device, inside the browser. Browsed text and images are not sent to a moderation server, and the project has no analytics or cloud inference backend.

In one sentence, the project is:

> **An on-device browser safety layer that moderates web content before the user has to see it.**

1. Problem statement

The problem I selected is involuntary exposure to harmful online content.

Social-media platforms, comment sections, search results, and general websites may contain hate speech, personal abuse, explicit images, violent imagery, or toxic memes. A user often sees this material before deciding whether to hide or report it. This is especially concerning for children and for people who want a safer browsing environment.

Existing platform moderation also has three limitations.

First, it is platform-specific. A setting on one social network does not protect the user on another website.

Second, moderation is normally controlled by the platform, so the user has limited control over thresholds and exceptions.

Third, many moderation systems send content to a server. That creates privacy, network-dependency, latency, and infrastructure-cost concerns.

SafeScroll addresses these issues at the browser level. It follows the user across websites, provides configurable local protection, and performs the main classification locally.

2. Project objectives

I defined six main objectives for the project.

The first objective was to detect multiple types of harmful content, not only text. The system therefore has separate layers for text toxicity, deterministic abusive-term matching, NSFW images, violence and gore, weapons, and OCR-based meme text.

The second objective was privacy. There is no server-side model inference, and model weights are packaged with the extension. The English OCR language data is downloaded as data and cached, but page content is not uploaded to a SafeScroll server because no such server exists.

The third objective was cross-site coverage. SafeScroll has specialized adapters for Instagram, X or Twitter, Reddit, and YouTube, plus a generic adapter for other websites, including search-result and article-style pages. The default scope is all HTTP and HTTPS websites, while the dashboard can restrict it to the core supported sites.

The fourth objective was responsiveness. The extension should not scan the complete DOM continuously or freeze scrolling.

The fifth objective was explainability. A verdict should show whether it came from text, a lexicon rule, an image model, violence detection, or OCR. The final decision uses an understandable rule instead of a hidden fusion model.

The sixth objective was user control. The project includes a master switch, per-site pause, configurable protection layers, a child lock, optional scroll lock, and a way to reveal or report a wrong verdict.

3. High-level architecture

SafeScroll uses the Chrome Manifest V3 extension architecture. It is divided into three main execution areas: the content script, the background service worker, and the offscreen document.

The data flow is:

```
Web page
   ↓
Content script: detect and extract visible content
   ↓
Background service worker: validate and relay messages
   ↓
Offscreen document: run local ML, OCR, fusion, and storage
   ↓
Content script: apply a text or media blur shield
```

The **content script** runs on the page. Its job is DOM interaction, not heavy machine learning. It discovers content units such as posts, comments, captions, search results, article sections, and media. It extracts relevant text and images, creates a stable identity for each unit, sends the content for classification, and renders the shield returned by the pipeline.

The **background service worker** acts mainly as a reliable message coordinator. Manifest V3 service workers are event-driven and can be suspended, so they are not suitable for keeping large models active. The service worker creates the offscreen document, performs a readiness handshake, relays requests, maintains a keep-alive alarm, and routes session events.

The **offscreen document** is the machine-learning engine room. It is a hidden extension page with the browser APIs needed for WebAssembly, canvas processing, Web Workers, and IndexedDB. It loads the text model, image models, and OCR worker; applies thresholds; combines verdicts; and stores results locally.

This separation keeps page-level code small while giving the ML pipeline a stable execution context.

4. How SafeScroll detects content on a live page

Modern websites do not load all posts at once. They use infinite scrolling and frequently recycle DOM nodes. Because of that, a one-time page scan is not enough.

SafeScroll combines two browser observers.

A **MutationObserver** detects content that has been inserted or changed. An **IntersectionObserver** prioritizes units near the viewport so the extension does not process the entire page unnecessarily. The current implementation debounces rapid changes, processes units in small batches, limits concurrent work, and uses a bounded queue.

The project also keeps a registry of content identities and verdicts. This prevents the same post from being classified repeatedly. When a website reuses a DOM element for a different post, SafeScroll compares the content identity so that an old verdict cannot be painted onto new content.

The extraction logic is adapter-based. Specialized adapters understand the DOM patterns of X, Instagram, Reddit, and YouTube. The generic adapter uses semantic and structural signals for other sites. All inner queries remain scoped to the current content unit, which prevents text from one post from being accidentally mixed with another post.

Long text is also handled deliberately. The XLM-R model has a token limit, so SafeScroll creates overlapping, sentence-aligned windows and takes the maximum toxicity score. It samples the full text rather than silently checking only the beginning of a long article or thread.

5. The moderation pipeline

The classification pipeline contains several independent protection layers.

Layer 1: deterministic lexicon

The first stage is a severity-graded English and Hinglish lexicon. It runs almost immediately and supports normalized matching, including some repeated-letter variations. High-severity terms produce a block verdict, while medium-severity terms produce a flag verdict.

This layer is deterministic and explainable. It is especially useful for direct profanity and known romanized Hinglish abuse that a general multilingual model may miss. If the lexicon already produces a block verdict, the text model does not need to repeat unnecessary work.

Layer 2: text-toxicity model

The main text model is **unitary/multilingual-toxic-xlm-roberta**, based on XLM-RoBERTa. It has approximately 278 million parameters.

I exported and packaged an INT8 ONNX version of the model. Quantization reduces the model from roughly 1.1 gigabytes in FP32 form to about 266 megabytes. The browser runs it through Transformers.js and ONNX Runtime Web using WebAssembly.

The model uses one output logit, so SafeScroll applies a sigmoid function manually to convert that logit into a toxicity probability. This detail is important because applying a softmax to one value would always produce 1.0 and make every result invalid.

For normal page text, a score of **0.90 or above** produces a block verdict. A score from **0.50 to below 0.90** produces a flag verdict. The two levels distinguish highly toxic content from milder negativity.

Layer 3: unsafe-image model

For sexual or NSFW imagery, SafeScroll uses **NSFWJS with MobileNetV2** through TensorFlow.js. The output classes are normalized before applying policy.

Porn and hentai scores at or above **0.70** produce a block, while scores at or above **0.50** produce a flag. The "sexy" class uses a stricter flag threshold of **0.85**, because lower values produced false positives on normal photographs. Neutral images and drawings are not flagged by that rule.

Layer 4: violence, gore, and weapon model

SafeScroll also contains a locally bundled, INT8, multi-label ResNet-50 model. It examines the violence, sensitive, and weapon outputs.

Violence at or above **0.82** produces a block and at or above **0.70** produces a flag. Gore uses the geometric mean of the violence and sensitive scores. This behaves like a soft AND condition: a medical image with a high sensitive score alone should not automatically be treated as gore. A weapon is flagged only at a high threshold of **0.88**.

The NSFW and violence models share a decoded bitmap and can run independently. If one detector fails, the other can still return a useful verdict.

Layer 5: OCR for meme text

An image can be visually safe while containing abusive text. To cover that case, SafeScroll uses Tesseract.js to read English text from images.

The OCR pipeline validates the image bytes, resizes when useful, converts the image to grayscale, and tries a small preprocessing ladder that includes inversion and Otsu thresholding. It also checks whether the result looks like real language instead of background noise.

OCR is expensive, so it runs outside the immediate text path, uses a token budget, caches outcomes per image, and has a bounded wait. Extracted meme text is sent through the same lexicon and toxicity logic, with more sensitive OCR-specific thresholds: **0.70 for block** and **0.35 for flag**.

Final fusion

The final fusion rule is intentionally simple:

> **Block if any layer blocks. Otherwise, flag if any layer flags. Otherwise, mark the content clean.**

This is an OR-fusion rule. It is easy to audit, and the stored reason tells us which layer triggered the decision.

6. Fast two-stage user experience

Text and media have very different execution costs. Text inference is normally faster, while image fetching and OCR can take much longer.

SafeScroll therefore uses a two-stage pipeline. It sends the text verdict to the page first. The content script can immediately remove the loading state or apply a text blur. A separate media request then runs image safety and OCR in the background. If the media result is more severe, the offscreen document sends a refined verdict and the existing shield is upgraded.

The extension also applies granular shields. A text-only finding blurs the relevant text container, while an image or OCR finding blurs the media container. It does not unnecessarily hide the entire post.

The shield provides a clear state, such as checking, flagged, or blocked. The user can choose "Show anyway," unless Child Lock requires the parent PIN, and can report an incorrect verdict.

7. User interface and controls

The popup provides quick controls. It contains the master enable switch, the current protection-engine state, blocked and flagged session counts, a current-site pause button, the active coverage scope, a pipeline self-test, and a link to the dashboard.

The dashboard provides a broader view. It shows totals, detection-source statistics, the latest twenty verdicts, and model information such as the model name, size, and load state. Users can select all-site or core-site coverage, manage paused websites, turn image protection and OCR on or off, clear the verdict cache, and configure Child Lock.

Child Lock requires a four-digit parent PIN before protected content can be revealed. Optional Scroll Lock can freeze the page when blocked content is detected until the correct PIN is entered.

The PIN implementation is intentionally described as a deterrent, not strong authentication. It currently stores a single SHA-256 hash without a salt, so a four-digit PIN could be brute-forced offline. A production version should use a salted, deliberately slow derivation method and retry controls.

8. Privacy, security, and local storage

Privacy is an architectural property of SafeScroll, not only a policy statement.

Text classification, NSFW classification, violence classification, OCR execution, thresholding, and verdict fusion run locally. The text and image model weights are bundled with the extension, and remote model loading is disabled. The project does not have an inference API, user-content database, or analytics endpoint.

Verdicts, image-result caching, and feedback are stored in IndexedDB. User preferences are stored through Chrome's extension storage. A pending-write mechanism improves durability if the service worker is suspended during a write.

Manifest V3 does not allow arbitrary remote code. The project bundles its JavaScript and WebAssembly runtime locally. Its Content Security Policy allows local scripts and the minimum WebAssembly permission needed by the ML runtimes, but it does not allow general unsafe JavaScript evaluation or remote scripts.

The extension needs broad host access because all-site moderation is a product requirement. To keep this transparent, the user can select core-site-only coverage and can pause individual domains.

9. Important engineering challenges and solutions

One major challenge was the Manifest V3 lifecycle. A background worker may be suspended, so large models cannot safely live there. I solved this by using a lightweight service worker and a dedicated offscreen ML document with an explicit ready and ping-pong handshake. The handshake has a hard timeout, so a failed engine does not silently classify everything as clean.

A second challenge was browser compatibility for local ML. Transformers.js initially mishandled extension URLs, browser cache behavior, WebAssembly workers, and one tokenizer field. I corrected the local model path, disabled incompatible caching and remote fallback, used a CSP-compatible single-threaded WASM setup, and patched the tokenizer metadata after comparing JavaScript and Python token IDs.

A third challenge was dynamic web-page behavior. Virtualized feeds reuse elements, lazy-load images, and repeatedly mutate small regions. Stable content identities, stale-result checks, media fingerprints, observer cleanup, deduplication, and bounded queues prevent repeated work and incorrect shields.

A fourth challenge was false positives. NSFWJS's "sexy" class runs relatively hot on ordinary images, so I introduced a higher class-specific threshold. For gore, I combined two signals so that sensitive medical content alone does not cause a block. For text, I separated high-confidence blocking from milder flagging.

A fifth challenge was OCR quality. Meme text can be small, inverted, stylized, or mixed with visual noise. The current pipeline uses validation, multiple preprocessing passes, plausibility scoring, source caching, and a bounded retry budget. It improves reliability, but stylized text remains an open research problem.

10. Evaluation and verification

The project uses several forms of verification.

For text quality, the current XLM-R model detected **8 out of 9** English identity-hate probe sentences, compared with **2 out of 9** for the earlier baseline. Clean examples in that probe remained clean. A phrase such as "this movie is stupid" may receive a flag, but it stays below the high-confidence block threshold.

For Hinglish, I created a fixed twenty-sentence acceptance set with ten toxic and ten clean examples. The acceptance gate requires both precision and recall of at least 0.70. Multiple fine-tuning experiments using MuRIL and multilingual MiniLM did not pass this gate. The best current production ML result reached precision **0.643**, recall **0.900**, and F1 **0.750**-better than earlier attempts, but still below the precision requirement. Therefore, I did not present a failed custom model as production-ready; the deterministic lexicon is the interim Hinglish protection.

The main root cause was insufficient high-quality Latin-script Hinglish slur supervision. Much of the available Hindi data was in Devanagari, while other datasets contained political hostility or noisy labels rather than the target behavior.

For software correctness, the current automated logic and regression checks pass. They cover text windowing, hashing, observer and registry behavior, site adapters, stale asynchronous results, OCR plausibility, lexicon boundaries, NSFW thresholds, violence and gore thresholds, UI hooks, and PIN-hash consistency.

The production build also completes successfully. A distribution verifier confirms that the packaged extension has no development-server dependency, and the remote-code audit currently reports **CLEAN**.

These checks do not mean the classifiers are perfect. They demonstrate that the implemented rules and software behavior match the design, while classifier accuracy remains probabilistic and data-dependent.

11. Limitations

I want to present the limitations clearly.

First, the extension bundle is large, mainly because the local text model is about 266 megabytes. This is the trade-off for local inference and no runtime model download.

Second, the Hinglish ML classifier has not passed the defined precision-and-recall gate. The lexicon helps with known expressions but cannot understand every context, spelling, or newly emerging term.

Third, OCR currently uses English recognition. Very stylized text, low-resolution images, and Devanagari meme text may be missed.

Fourth, the violence model's published evaluation set is relatively small, so its results should be treated as probabilistic filtering rather than a safety guarantee.

Fifth, broad website coverage is difficult because DOM structures change. The generic detector provides resilience, but specialized adapters may still need maintenance when major sites redesign their pages.

Sixth, the Child Lock PIN protects against casual bypass, not a determined local attacker.

Finally, on-device inference consumes memory and CPU. The current system limits concurrency and prioritizes near-viewport content, but older devices may still experience slower classification.

12. Future scope

The first future improvement is a smaller and faster text model, provided it can retain the current English quality. Distillation, additional quantization, WebGPU execution, or a cascade in which a small model handles easy examples could reduce latency and package size.

The second improvement is a properly supervised Hinglish model. The next training cycle should use an official Latin-script code-mixed dataset such as the relevant HASOC track, with careful label auditing and a held-out acceptance set.

The third improvement is multilingual OCR, especially Devanagari and robust stylized-text recognition. A more advanced OCR fallback could run only when the first pass has low confidence.

The fourth improvement is stronger parental security using a salted password derivation function, retry delays, and protected settings changes.

The fifth improvement is broader evaluation. I would build larger, balanced test sets for toxic text, normal images, explicit images, violence, gore, medical imagery, memes, and cross-site DOM behavior. Precision, recall, false-positive rate, latency, memory use, and energy impact should all be reported by device category.

The sixth improvement is a human-reviewed feedback loop. Feedback is already stored locally; a future workflow could let the user export selected, consented, anonymized examples for manual dataset curation. Automated retraining should not occur without review because incorrect feedback can poison the model.

13. Demonstration script

> ***[Open the Chrome extensions page and show that SafeScroll is loaded from the production `dist` folder.]***

I will now demonstrate the system.

First, this is the SafeScroll popup. The master toggle enables or disables protection. The status area shows whether the local models are loading or ready. The counters show blocked and flagged decisions in the current session. I can also pause only the current website without disabling protection everywhere.

> ***[Open a clean test page or normal feed.]***

On this page, clean content remains visible. SafeScroll does not replace the website; it only adds a lightweight protection layer.

> ***[Open or inject a high-confidence toxic-text example.]***

This text crosses the block threshold. Notice that only the text region is blurred and the reason is shown on the shield. The rest of the post remains usable.

> ***[Show a mild example.]***

This second example is negative but less severe. It receives a flag rather than a block, showing the purpose of the two-tier design.

> ***[Show an unsafe-image or toxic-meme test fixture.]***

For media, the image model and violence model inspect the visual content. OCR separately checks whether a meme contains toxic text. If a media result is more severe than the first text verdict, the page receives a refined verdict and updates the shield.

> ***[Click "Show anyway."]***

The user remains in control and can reveal the content. When Child Lock is active, this action requires the parent PIN.

> ***[Open the dashboard.]***

The dashboard shows session statistics, recent verdicts, model load information, coverage settings, protection-layer controls, paused websites, Child Lock settings, and cache maintenance.

> ***[Optionally run the popup pipeline self-test.]***

This self-test sends a known toxic sentence through the real local text model. It verifies that the model and message path are working, rather than only testing the interface.

That completes the demonstration.

14. Conclusion

To conclude, SafeScroll demonstrates that meaningful content moderation can run directly inside a browser extension while preserving user privacy and control.

The project combines browser engineering, on-device machine learning, DOM analysis, model quantization, OCR, local storage, security constraints, and user-interface design. Its main contribution is not only a classifier. It is a complete, explainable pipeline that discovers content on changing web pages, classifies multiple modalities locally, and applies a granular safety response in real time.

I have also treated failed experiments as engineering evidence. The Hinglish models were not shipped because they did not pass the acceptance gate, and the limitation is documented with a concrete path forward.

The current version builds successfully, passes its logic and regression checks, and passes the production remote-code audit. Its remaining limitations-bundle size, multilingual coverage, OCR quality, model validation, and PIN strength-form the next development roadmap.

Thank you. I am ready for your questions and feedback.

Likely Questions and Suggested Answers

Why did you choose a browser extension?

The browser is the point where the user consumes content across many websites. An extension can provide one consistent safety layer without waiting for every platform to implement the same controls.

Why run the models locally instead of on a server?

Local inference protects browsing privacy, avoids per-request server cost, reduces dependence on network availability, and removes the need to maintain a user-content backend. The trade-offs are a larger package and more CPU and memory use on the client.

Why use an offscreen document?

Manifest V3 service workers are temporary and do not provide a normal DOM environment. The offscreen document gives the ML pipeline a hidden page with WebAssembly, canvas, worker, and IndexedDB support while the service worker remains lightweight.

Why use both a lexicon and a machine-learning model?

They solve different problems. The lexicon is immediate, deterministic, and strong for known abusive terms and spelling patterns. The ML model can recognize context and unseen phrasing. Combining them improves coverage and keeps rule-based detections explainable.

Why is the fusion rule an OR condition?

Each layer detects a different risk. A safe image does not cancel toxic text, and clean visible text does not cancel an unsafe image or abusive meme caption. OR fusion also produces a clear reason trail and avoids the need for another training dataset for a learned fusion model.

How do you control false positives?

I use separate block and flag thresholds, class-specific image thresholds, a high-confidence weapon rule, a joint violence-sensitive gore score, whole-word lexicon matching, OCR plausibility checks, and explicit clean test cases. Users can reveal content and report incorrect verdicts.

What happens when a model fails to load?

The stages are isolated. One image detector failing does not automatically disable the other detector or the text path. The readiness state and diagnostics expose failures, and the offscreen handshake has a hard timeout so the system does not silently claim success.

How does SafeScroll avoid slowing down scrolling?

It observes content near the viewport, debounces mutations, processes small batches, limits concurrent classifications, caches verdicts, deduplicates media work, serializes single-threaded WASM inference, and moves slower media and OCR work outside the first text response.

What is quantization?

Quantization stores model weights with lower numerical precision. The text model uses INT8 weights instead of FP32 weights, reducing its size by roughly four times and making browser deployment more practical while preserving useful classification behavior.

Does the extension work completely offline?

The bundled JavaScript, WebAssembly runtimes, text model, NSFW model, and violence model work locally. Tesseract's English trained-data file may need to be downloaded and cached the first time, and a website obviously needs its normal network connection. The project does not require a SafeScroll inference server.

Is the Child Lock secure?

It prevents casual bypass, but it is not strong security. A four-digit PIN with a single unsalted SHA-256 hash has a small search space. A production version should use a salt, a slow derivation function, retry limits, and stronger settings protection.

Why did the Hinglish experiments fail?

The main issue was data mismatch. Much of the available Hindi hate-speech data used Devanagari, while the target was romanized Hindi-English code-mixing. Other corpora contained political hostility or noisy labels rather than enough direct slur examples. The experiments showed that more training on mismatched data does not solve the problem.

What is the main innovation in this project?

The strongest contribution is the end-to-end, privacy-first system design: cross-site DOM detection, multiple local classifiers, OCR, explainable fusion, streaming verdict refinement, granular blurring, durable local state, and parental controls working together inside Manifest V3 constraints.

How would you deploy this as a real product?

I would first reduce the package size, strengthen the PIN design, expand benchmark datasets, test on low-end devices, add automated cross-browser regression tests, complete accessibility and privacy reviews, and then package the signed production build for browser-store review.

60-second backup summary

SafeScroll is a Chrome Manifest V3 extension that protects users from harmful web content using machine learning on the device. It detects toxic text, direct abusive terms, NSFW images, violence, gore, weapons, and toxic meme text. The content script discovers posts and comments near the viewport; the service worker coordinates messages; and an offscreen document runs Transformers.js, ONNX Runtime, TensorFlow.js, and Tesseract. The system uses explainable OR fusion and applies granular text or media blur shields. It supports all websites through core-site and generic adapters, includes per-site pause and parental controls, and stores verdicts locally. The current build and regression checks pass, while known limitations-including a large text model, incomplete Hinglish ML coverage, OCR limitations, and a weak four-digit PIN design-are explicitly documented as future work.